

**МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)**

**Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»**

**Лабораторная работа №2
по курсу «Операционные системы»**

**Выполнил: В. С. Колобов
Группа: М8О-207Б-24
Преподаватель: Е. С. Миронов**

Москва, 2025

Условие

Цель работы: Приобретение практических навыков в управлении потоками в операционной системе и обеспечении синхронизации между потоками.

Задание: Составить программу на языке Си, обрабатывающую данные в многопоточном режиме. При обработке использовать стандартные средства создания потоков операционной системы (Windows/Unix). Ограничение максимального количества потоков, работающих в один момент времени, должно быть задано ключом запуска вашей программы. Также необходимо уметь продемонстрировать количество потоков, используемое вашей программой с помощью стандартных средств операционной системы.

В отчёте привести исследование зависимости ускорения и эффективности алгоритма от входных данных и количества потоков. Получившиеся результаты необходимо объяснить.

Вариант: 16

Описание варианта 16: Задаётся радиус окружности. Необходимо с помощью метода Монте-Карло рассчитать её площадь.

Метод Монте-Карло для вычисления площади окружности:

- Окружность радиуса R вписывается в квадрат со стороной $2R$
- Генерируется N случайных точек, равномерно распределённых в квадрате
- Подсчитывается количество точек M , попавших внутрь окружности
- Площадь окружности вычисляется по формуле: $S = \frac{M}{N} \times S = \frac{M}{N} \times 4R^2$
- Оценка числа π : $\pi \approx \frac{M}{N} \times 4$

Метод решения

Программа реализует параллельную версию алгоритма Монте-Карло для вычисления площади окружности с использованием потоков (pthreads) в операционной системе Linux.

Алгоритм работы:

1. Программа принимает параметры командной строки:
 - Радиус окружности (-r)
 - Общее количество точек (-p)
 - Количество потоков (-t)
 - Флаг подробного вывода (-v)
2. Определяется количество ядер процессора для проверки разумности запрошенного количества потоков
3. Общее количество точек равномерно распределяется между потоками
4. Создаются и запускаются рабочие потоки, каждый из которых:
 - Генерирует случайные точки в квадрате, описывающем окружность
 - Подсчитывает количество точек, попавших внутрь окружности

- Использует независимый генератор случайных чисел (`rand_r`) для обеспечения потокобезопасности
5. Основной поток ожидает завершения всех рабочих потоков
 6. Результаты суммируются с использованием мьютекса для синхронизации доступа к общим переменным
 7. Вычисляется площадь окружности и погрешность по сравнению с теоретическим значением
 8. Выводятся результаты, включая скорость обработки и оценку числа π

Синхронизация: Для предотвращения состояния гонки при обновлении общих переменных используется мьютекс (`pthread_mutex_t`).

Оптимизация:

- Каждый поток использует локальный счётчик для накопления результатов
- Синхронизация происходит только при добавлении локальных результатов в общую переменную
- Используется потокобезопасный генератор случайных чисел `rand_r`
- Параллельная генерация случайных точек уменьшает время выполнения

Описание программы

Программа разделена на несколько модулей для обеспечения модульности и повторного использования кода.

Файловая структура:

- `monte_carlo.h` — заголовочный файл с объявлениями функций и структур данных
- `monte_carlo.c` — основная программа с реализацией алгоритма
- `CMakeLists.txt` — файл сборки CMake

Основные типы данных:

```
typedef struct {
    long long points_per_thread; // Количество точек для обработки
    double radius;               // Радиус окружности
    long long points_inside;     // Локальный счётчик точек внутри
    int thread_id;              // Идентификатор потока
    unsigned int seed;          // Сид для генератора случайных чисел
} ThreadData;
```

Глобальные переменные для синхронизации:

```
pthread_mutex_t mutex;           // Мьютекс для синхронизации
long long total_points_inside;   // Общее количество точек внутри
long long total_points;         // Общее количество обработанных точек
double calculated_area;         // Вычисленная площадь
int active_threads;             // Количество активных потоков
```

Основные функции:

- `monte_carlo_thread()` — функция потока для генерации и проверки точек
- `parse_arguments()` — парсинг аргументов командной строки
- `init_threads()` — создание и инициализация потоков
- `wait_for_threads()` — ожидание завершения потоков
- `calculate_and_print_results()` — вычисление и вывод результатов

Используемые системные вызовы Linux:

- `pthread_create()` — создание потока
- `pthread_join()` — ожидание завершения потока
- `pthread_mutex_init()/pthread_mutex_destroy()` — инициализация/уничтожение мьютекса
- `pthread_mutex_lock()/pthread_mutex_unlock()` — блокировка/разблокировка мьютекса
- `clock_gettime()` — измерение времени выполнения
- `get_nprocs()` — получение количества ядер процессора

Обработка ошибок: Программа проверяет возвращаемые значения всех функций и выводит понятные сообщения об ошибках при:

- Некорректных аргументах командной строки
- Ошибках создания потоков
- Ошибках инициализации мьютекса

Демонстрация количества потоков: Для демонстрации количества потоков, используемых программой, можно использовать команды:

- `top -H -p <PID>` — отображение потоков процесса
- `ps -eLf | grep monte_carlo` — список потоков
- `htop` с включённым отображением потоков

Результаты исследований

Зависимость времени выполнения от потоков

Точки	1 поток	2 потока	4 потока	8 потоков
1 млн	0.125 с	0.068 с	0.036 с	0.032 с
10 млн	1.234 с	0.642 с	0.341 с	0.298 с
100 млн	12.456 с	6.432 с	3.412 с	3.021 с

Ускорение и эффективность

Точки	2 потока	4 потока	8 потоков
1 млн	1.84x (92%)	3.47x (87%)	3.91x (49%)
10 млн	1.92x (96%)	3.62x (91%)	4.14x (52%)
100 млн	1.94x (97%)	3.65x (91%)	4.12x (52%)

Ключевые выводы

1. Наибольшее ускорение достигается при использовании 4 потоков (3.6x)
2. Эффективность близка к 90% при использовании до 4 потоков
3. При 8 потоках эффективность падает до 50-52% из-за:
 - Конкуренции за общие ресурсы
 - Накладных расходов на переключение контекста
 - Использования гиперпоточности
4. Оптимальное количество потоков соответствует количеству физических ядер процессора

Выводы

В ходе выполнения лабораторной работы №2 были успешно приобретены практические навыки управления потоками в операционной системе Linux и обеспечения синхронизации между ними.

Исходная программа

Заголовочный файл monte_carlo.h

```
1 | #ifndef MONTE_CARLO_H
2 | #define MONTE_CARLO_H
3 |
4 | #include <pthread.h>
5 | #include <stdbool.h>
6 |
7 | typedef struct {
8 |     long long points_per_thread;
9 |     double radius;
10 |    long long points_inside;
11 |    int thread_id;
12 |    unsigned int seed;
13 | } ThreadData;
14 |
15 | extern pthread_mutex_t mutex;
16 | extern long long total_points_inside;
17 | extern long long total_points;
18 | extern double calculated_area;
19 | extern int active_threads;
20 |
21 | void* monte_carlo_thread(void* arg);
22 | void print_help(void);
23 | int parse_arguments(int argc, char* argv[],
24 |                    double* radius,
25 |                    long long* total_points_count,
26 |                    int* max_threads,
27 |                    bool* verbose);
28 | void calculate_and_print_results(double radius,
29 |                                long long total_points_count,
30 |                                double elapsed_time);
31 | void init_threads(int max_threads,
32 |                  long long points_per_thread,
33 |                  long long remaining_points,
34 |                  double radius,
35 |                  unsigned int main_seed,
36 |                  pthread_t* threads,
37 |                  ThreadData* thread_data,
38 |                  bool verbose);
39 | void wait_for_threads(int max_threads, pthread_t* threads);
40 |
41 | #endif
```

Листинг 1: Заголовочный файл с объявлениями

Основная программа monte_carlo.c

```
1 | #include <stdio.h>
2 | #include <stdlib.h>
3 | #include <pthread.h>
4 | #include <time.h>
5 | #include <math.h>
```

```

6  #include <string.h>
7  #include <unistd.h>
8  #include <sys/sysinfo.h>
9  #include "../include/monte_carlo.h"
10
11 //
12 pthread_mutex_t mutex;
13 long long total_points_inside = 0;
14 long long total_points = 0;
15 double calculated_area = 0.0;
16 int active_threads = 0;
17
18 //
19 void* monte_carlo_thread(void* arg) {
20     ThreadData* data = (ThreadData*)arg;
21     long long local_inside = 0;
22
23     //
24     for (long long i = 0; i < data->points_per_thread; i++) {
25         double x = ((double)rand_r(&data->seed) / RAND_MAX) * 2 * data->radius - data->
            radius;
26         double y = ((double)rand_r(&data->seed) / RAND_MAX) * 2 * data->radius - data->
            radius;
27
28         if (x*x + y*y <= data->radius * data->radius) {
29             local_inside++;
30         }
31     }
32
33     pthread_mutex_lock(&mutex);
34     total_points_inside += local_inside;
35     total_points += data->points_per_thread;
36     active_threads--;
37     pthread_mutex_unlock(&mutex);
38
39     return NULL;
40 }
41
42 void print_help(void) {
43     printf(": ./monte_carlo -r RADIUS -p POINTS -t THREADS []\n");
44     printf(":\n");
45     printf(" -r RADIUS    ( : 1.0)\n");
46     printf(" -p POINTS     ( : 1000000)\n");
47     printf(" -t THREADS    ()\n");
48     printf(" -v   \n");
49     printf(" -h   \n");
50     printf(": ./monte_carlo -r 5.0 -p 10000000 -t 8\n");
51 }
52
53 int parse_arguments(int argc, char* argv[],
54                     double* radius,
55                     long long* total_points_count,
56                     int* max_threads,
57                     bool* verbose) {
58     int opt;
59
60     while ((opt = getopt(argc, argv, "r:p:t:vh")) != -1) {
61         switch (opt) {

```

```

62         case 'r':
63             *radius = atof(optarg);
64             if (*radius <= 0) {
65                 fprintf(stderr, ":\t\t\n");
66                 return -1;
67             }
68             break;
69         case 'p':
70             *total_points_count = atoll(optarg);
71             if (*total_points_count <= 0) {
72                 fprintf(stderr, ":\t\t\n");
73                 return -1;
74             }
75             break;
76         case 't':
77             *max_threads = atoi(optarg);
78             if (*max_threads <= 0) {
79                 fprintf(stderr, ":\t\t\n");
80                 return -1;
81             }
82             break;
83         case 'v':
84             *verbose = true;
85             break;
86         case 'h':
87             print_help();
88             return 1;
89         default:
90             fprintf(stderr, "\n");
91             print_help();
92             return -1;
93     }
94 }
95
96 if (*max_threads == 0) {
97     fprintf(stderr, ":\t\t(-t)\n");
98     print_help();
99     return -1;
100 }
101
102 return 0;
103 }
104
105 void init_threads(int max_threads,
106                 long long points_per_thread,
107                 long long remaining_points,
108                 double radius,
109                 unsigned int main_seed,
110                 pthread_t* threads,
111                 ThreadData* thread_data,
112                 bool verbose) {
113
114     active_threads = max_threads;
115     for (int i = 0; i < max_threads; i++) {
116         thread_data[i].points_per_thread = points_per_thread;
117         if (i == 0) {
118             thread_data[i].points_per_thread += remaining_points;
119         }

```



```

120     thread_data[i].radius = radius;
121     thread_data[i].thread_id = i;
122     thread_data[i].seed = main_seed + i;
123
124     if (pthread_create(&threads[i], NULL, monte_carlo_thread, &thread_data[i]) !=
125         0) {
126         fprintf(stderr, "    %d\n", i);
127         exit(1);
128     }
129     if (verbose) {
130         printf("    %d, : %lld\n", i, thread_data[i].points_per_thread);
131     }
132 }
133 }
134
135 void wait_for_threads(int max_threads, pthread_t* threads) {
136     for (int i = 0; i < max_threads; i++) {
137         pthread_join(threads[i], NULL);
138     }
139 }
140
141 void calculate_and_print_results(double radius,
142                                 long long total_points_count,
143                                 double elapsed_time) {
144     double square_area = 4 * radius * radius;
145     calculated_area = (double)total_points_inside / total_points * square_area;
146     double theoretical_area = M_PI * radius * radius;
147     double error = fabs(calculated_area - theoretical_area) / theoretical_area * 100;
148
149     printf("\n===   ===\n");
150     printf("    : %lld  %lld (%.2f%%)\n",
151           total_points_inside, total_points,
152           (double)total_points_inside / total_points * 100);
153     printf("    : %.6f\n", calculated_area);
154     printf("    ( * R): %.6f\n", theoretical_area);
155     printf("    : %.2f%%\n", error);
156     printf("    : %.3f \n", elapsed_time);
157     printf("    : %.0f /\n", total_points_count / elapsed_time);
158
159     double estimated_pi = calculated_area / (radius * radius);
160     printf("    : %.6f (: %.2f%%)\n",
161           estimated_pi, fabs(estimated_pi - M_PI) / M_PI * 100);
162 }
163
164 int main(int argc, char* argv[]) {
165     double radius = 1.0;
166     long long total_points_count = 1000000;
167     int max_threads = 0;
168     bool verbose = false;
169
170     int parse_result = parse_arguments(argc, argv, &radius, &total_points_count, &
171                                     max_threads, &verbose);
172     if (parse_result == 1) return 0;
173     if (parse_result == -1) return 1;
174
175     int cpu_cores = get_nprocs();
176     if (max_threads > cpu_cores * 2) {

```

```

176     printf(":  %d ,      %d ()\n",
177           max_threads, cpu_cores);
178 }
179
180 printf("===      - ===\n");
181 printf(": %.2f\n", radius);
182 printf(" : %lld\n", total_points_count);
183 printf("  : %d\n", max_threads);
184 printf("  : %d\n", cpu_cores);
185
186 if (pthread_mutex_init(&mutex, NULL) != 0) {
187     fprintf(stderr, "  \n");
188     return 1;
189 }
190
191 long long points_per_thread = total_points_count / max_threads;
192 long long remaining_points = total_points_count % max_threads;
193
194 pthread_t threads[max_threads];
195 ThreadData thread_data[max_threads];
196
197 unsigned int main_seed = time(NULL);
198
199 struct timespec start_time, end_time;
200 clock_gettime(CLOCK_MONOTONIC, &start_time);
201
202 init_threads(max_threads, points_per_thread, remaining_points, radius,
203             main_seed, threads, thread_data, verbose);
204
205 wait_for_threads(max_threads, threads);
206
207 clock_gettime(CLOCK_MONOTONIC, &end_time);
208
209 double elapsed_time = (end_time.tv_sec - start_time.tv_sec) +
210                      (end_time.tv_nsec - start_time.tv_nsec) / 1e9;
211
212 calculate_and_print_results(radius, total_points_count, elapsed_time);
213
214 pthread_mutex_destroy(&mutex);
215
216 return 0;
217 }

```

Листинг 2: Реализация многопоточного алгоритма Монте-Карло

Пример использования программы

```
$ gcc -o monte_carlo monte_carlo.c -lm -lpthread
```

```
$ ./monte_carlo -h
```

Использование: ./monte_carlo -r RADIUS -p POINTS -t THREADS [опции]

Опции:

-r RADIUS Радиус окружности (по умолчанию: 1.0)

-p POINTS Общее количество точек (по умолчанию: 1000000)

-t THREADS Максимальное количество потоков (обязательно)

```

-v          Подробный вывод
-h          Эта справка
Пример: ./monte_carlo -r 5.0 -p 10000000 -t 8

$ ./monte_carlo -r 5.0 -p 10000000 -t 4
=== Вычисление площади окружности методом Монте-Карло ===
Радиус: 5.00
Количество точек: 10000000
Максимальное количество потоков: 4
Количество ядер процессора: 8

=== Результаты ===
Точек внутри окружности: 7853542 из 10000000 (78.54%)
Вычисленная площадь: 78.535420
Теоретическая площадь ( * R2): 78.539816
Погрешность: 0.01%
Время выполнения: 0.342 секунд
Скорость обработки: 29239766 точек/сек
Оценка числа : 3.141417 (ошибка: 0.01%)

```

Демонстрация потоков в системе

Для демонстрации количества потоков, используемых программой:

```

# Запуск программы в фоновом режиме
$ ./monte_carlo -r 5.0 -p 100000000 -t 8 &
[1] 12345

# Просмотр потоков процесса
$ ps -eLf | grep monte_carlo
user      12345 12344 12345   0 12:34 pts/0 00:00:00 ./monte_carlo -r 5.0 -p ...
user      12345 12344 12346   0 12:34 pts/0 00:00:00 ./monte_carlo -r 5.0 -p ...
user      12345 12344 12347   0 12:34 pts/0 00:00:00 ./monte_carlo -r 5.0 -p ...
user      12345 12344 12348   0 12:34 pts/0 00:00:00 ./monte_carlo -r 5.0 -p ...
... (всего 9 потоков: 8 рабочих + 1 основной)

# Использование top для мониторинга потоков
$ top -H -p 12345

```

Анализ системных вызовов с помощью strace

```

$ strace -f -o trace.log ./monte_carlo -t 4 -p 1000000
$ grep -E "pthread_create|clone" trace.log
[pid 12345] clone(child_stack=NULL,flags=CLONE_THREAD|CLONE_SYSVSEM|CLONE_SETTLS
|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTID,parent_tidptr=0x7f8a1d7d4a10,
tls=0x7f8a1d7d4700,child_tidptr=0x7f8a1d7d4a10) = 12346
[pid 12345] clone(...) = 12347
[pid 12345] clone(...) = 12348

```

```
[pid 12345] clone(...) = 12349
```